

CSCE 465 HW 2 Report

GitHub Repository

[Repo](#)

Task 0 - Lab Preparation

```
> python3 --version
Python 3.12.3

> openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)

> pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to
Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors
<cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/jacob/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel,
Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/jacob/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:

> openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
```

Task 1 - Demonstrate why encryption alone is insufficient

The Design

I created two classes: `Sender` and `Receiver`. The `main` function generates a key and initial value and initializes each class with them. This effectively represents the secret key exchange, so the sender may encrypt its message, and the receiver may decrypt the cipher with the same

key. The relay function represents a man in the middle. `main` routes the sender's encrypted message through the relay function, to the receiver. `main` also facilitates the replay attack by replaying the encrypted message from sender to the receiver twice.

What Happened

In the relay experiment, the sender sends its encrypted message to the receiver. This gets routed through the relay, which modifies the select bits and relays the message to the receiver. When the receiver decrypts the message, the action has been changed from `READ` to `PWND`.

The original message

```
{"action": "READ", "path": "notes.txt"}
```

The message received

```
{"action": "PWND", "path": "notes.txt"}
```

During the replay attack experiment, the sender sends its message once, and the receiver receives and decrypts two messages. Both decrypted messages are the same.

Root Cause

In AES-CTR encryption, changing a bit predictably changes the bit in the ciphertext. This fact can be used to attack the integrity of the transmitted message. In the relay experiment, the relay function acts as a MITM. The `READ` command and its offset are known, so an attacker can use the properties of XOR to alter the ciphertext to decrypt to a desired message. Bitwise XOR of the original message with the desired message creates a delta. XOR the ciphertext with this delta returns the ciphertext version of the desired message.

```
READ --> PWND
```

```
Original bytes:  52 45 41 44
```

```
Desired bytes:   50 57 4e 44
```

```
XOR difference:  02 12 0f 00
```

```
8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939
```

```
8ec2b866ced37843798ab44ab8cf39696e4bff10435089f748741b9d6db359d941df8939
```

AES-CTR encryption is also vulnerable to replay attacks, because there is no authentication of who sent the message or when it was sent. A receiver cannot tell if the message is valid or if it has been duplicated from an earlier transmission.

```

(.venv) jacob@hw1-agent:~/csce465-agentsec/hw2$ python baseline_ctr.py

Sender and Receiver initialized with same key and initial value.

=====
Message Integrity Demonstration
=====
SENDER
-----
Plaintext Message: {"action": "READ", "path": "notes.txt"}
Encrypted Message: 8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939

RELAY
-----
Received Ciphertext: 8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939

XOR Relation (READ --> PWND):
  Original bytes: 52 45 41 44
  Desired bytes: 50 57 4e 44
  XOR difference: 02 12 0f 00

Modified Ciphertext: 8ec2b866ced37843798ab44ab8cf39696e4bff10435089f748741b9d6db359d941df8939

RECEIVER
-----
Received Ciphertext: 8ec2b866ced37843798ab44ab8cf39696e4bff10435089f748741b9d6db359d941df8939
Decrypted Message: {"action": "PWND", "path": "notes.txt"}

=====
Replay Demonstration
=====
SENDER
-----
Plaintext Message: {"action": "READ", "path": "notes.txt"}
Encrypted Message: 8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939

RECEIVER
-----
Received Ciphertext: 8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939
Decrypted Message: {"action": "READ", "path": "notes.txt"}
Received Ciphertext: 8ec2b866ced37843798ab448aac039696e4bff10435089f748741b9d6db359d941df8939
Decrypted Message: {"action": "READ", "path": "notes.txt"}

```

Task 2 - Authenticated finite-field Diffie-Hellman setup

Overview

In this task, I implemented an authenticated key exchange protocol between two parties using finite-field Diffie-Hellman. Each endpoint was initialized with long-term RSA signing keys and the same Diffie-Hellman values loaded from the group file. A handshake was initiated in which each party sent a hello message containing public information, built a canonical transcript using the received hello, and computed the transcript hash. Each endpoint signs the transcript hash, which is then verified by the other. After authentication the session keys are derived and returned to be used for secure communication.

Endpoint Initialization

An endpoint is defined by the `Party` class. Each endpoint is initialized with the following information

```

Party
  Identity
  Role
  DH Parameters
  RSA Signing Keys

```

```
Trusted Peers
  Peer Identity: Peer RSA Public Signing Key
Peer Role
```

Handshake Design & Considerations

The handshake protocol was designed to reconstruct the exchange of keys without exposing secrets over the wire. The protocol implements the following steps:

01 Hello

Each endpoint generates a fresh Diffie-Hellman public and private key pair, as well as the 16-byte nonce. It then sends a hello message including its identity, public DH signing key, and the nonce.

02 Build Canonical Transcript

Using the received Hello message, each endpoint builds a canonical transcript using the required field order and encoding. While designing my protocol, I initially had the handshake build a central transcript, which was then sent to the endpoints for hashing and signing. However, this is a vulnerable design. Tampering either Hello message results in an altered transcript, but neither party can recognize this. The more secure design is to have each endpoint build a transcript individually using the received Hello message and its known values. If either Hello message is tampered with, the transcripts will differ, and signature authentication will fail.

03 Sign Transcript

Each party hashes the transcript, signs it with their role, and sends the signature to its peer. By signing the transcript hash with its role, we are effectively preventing a reflection attack. A party knows its role and knows what role to expect from a peer. If a MITM intercepts a signature and reflects it back, the party will identify the unexpected role and signature verification will fail.

04 Verify Peer Signature

Upon receiving a peer's signature, the endpoint verifies it against the expected signature. Each party knows the other peer's role, public signing key, and has computed the transcript hash itself. Using these, the part constructs the signature it expects from its peer and compares it against that received. Any mismatch results in an authentication failure and the signature is rejected. This is the benefit of initializing each endpoint with the knowledge of the other party's public information and constructing the transcripts individually. Any tampering with the messages sent between the two parties will result in an invalid signature and authentication will fail. This is what makes this Diffie-Hellman key exchange protocol resistant to MITM attacks.

05 Derive Session Keys & Establish Sessions

After mutual authentication of the signatures, the parties individually derive the session keys. At this point, each party has received and verified the required information needed for these keys. The session keys are derived using the required KDF. The keys generated by both sides are

identical, and the authenticated sessions with their keys are returned to be used for secure communication between the parties.

```
(.venv) jacob@hw1-agent:~/csce465-agentsec/hw2$ python handshake.py
[*] Loaded DH parameters from ffdhe3072.pem (group ID: b'ffdhe3072')
[*] Generated long-term RSA signing keys for Gateway and Node
[*] Initialized Gateway and Node

Beginning handshake between Gateway and Node
=====
[*] gateway generated fresh DH key pair and 16-byte nonce
[*] node generated fresh DH key pair and 16-byte nonce
[*] gateway constructed transcript
[*] node constructed transcript
[*] Transcript hashes match
[*] gateway signed transcript hash
[*] node signed transcript hash
[*] gateway verified Gateway RSA-PSS signature
[*] node verified Gateway RSA-PSS signature
[*] gateway derived session keys
[*] node derived session keys
[*] Session IDs match: bc3ec8b683cc363a
[*] Gateway send keys match Node receive keys
[*] Node send keys match Gateway receive keys
[*] Sessions established successfully
```

Importance of Signatures

Signatures provide authentication. By signing and verifying a message, the endpoint is ensuring a message is from who the source they expect. If the signature step is removed, the handshake protocol becomes vulnerable to a MITM attack. A MITM could create a session with each party while leading them to believe they have established a connection with each other. The bad actor simply needs to intercept the Hello messages and inject its information. The endpoints can build the transcripts internally, but these are never verified with their peer's signature. A connection between the MITM and an endpoint is established, and the attacker can encrypt, decrypt, and tamper with messages between the two parties.

Task 3 - Encrypt-then-MAC record layer

Overview

Task three implements a secure record layer on top of the Diffie-Hellman handshake implemented in Task 2. The handshake establishes a session corresponding to each endpoint that contains separate session keys, directions, and sequence numbers. The record layer builds on top of these sessions. `seal` converts plain text into an encrypted record using the required

format. `open_record` authenticates the record and decrypts it into plaintext using the session's receive key. The record format used is as follows:

```
record = header || IV || ciphertext || tag
```

seal Design

The `seal` function takes a session, plaintext, and a message type and returns the record described as before. The session provides the encryption keys established during the handshake. The implementation is as follows:

1. Retrieve the send sequence number from the session.
2. Construct the IV from the session id and send sequence number.
3. Construct the header using the required format
4. Encrypt the plaintext with AES-256-CTR using the session's sending encryption key.
5. Compute the HMAC tag of the header, IV, and ciphertext using the session's sending MAC key.
6. Assemble the final record as described.
7. Increment the session's send sequence number.

By computing the HMAC tag over the header, IV, and ciphertext, an endpoint is able to check if any of the tagged data has been tampered by comparing the hash themselves. Any modifications to the tagged data or HMAC tag will cause authentication to fail.

open_record Design

The `open_record` function takes a session and record, and returns the message type and decrypted plain text. The implementation is as follows:

1. Unpack header and validate the record size.
2. Parse the record for the IV, ciphertext, and authentication tag.
3. Verify the HMAC tag by computing it using the session's receiving MAC key and comparing it to the tag.
4. Validate the record metadata.
5. Decrypt the ciphertext with AES-256-CTR using the session's receiving encryption key.
6. Increment the session's receive sequence number.
7. Return the message type and plaintext.

```
(.venv) jacob@hw1-agent:~/csce465-agentsec/hw2$ python secure_record.py
[*] Loaded DH parameters from ffdhe3072.pem (group ID: b'ffdhe3072')
[*] Generated long-term RSA signing keys for Gateway and Node
[*] Initialized Gateway and Node

Beginning handshake between Gateway and Node
=====
[*] gateway generated fresh DH key pair and 16-byte nonce
[*] node generated fresh DH key pair and 16-byte nonce
[*] gateway constructed transcript
[*] node constructed transcript
[*] Transcript hashes match
[*] gateway signed transcript hash
[*] node signed transcript hash
[*] gateway verified Gateway RSA-PSS signature
[*] node verified Gateway RSA-PSS signature
[*] gateway derived session keys
[*] node derived session keys
[*] Session IDs match: 7293bdc9a61f77c
[*] Gateway send keys match Node receive keys
[*] Node send keys match Gateway receive keys
[*] Sessions established successfully
```

```
=====
SEALING RECORD
-----
Plaintext: b'Hello, Node'
Plaintext length: 11 bytes
Message type: 1
Send sequence: 0
=====
[*] Constructed header
    Version: 1
    Direction: 1
    Sequence: 0
    Message type: 1
    Ciphertext length: 11 bytes
[*] Header length: 15 bytes
[*] IV length: 16 bytes
[*] Ciphertext: 6c25eaca11854b9e5476e8
[*] Ciphertext length: 11 bytes
[*] MAC input length: 42 bytes
[*] Tag length: 32 bytes
[*] Total record length: 74 bytes
[*] Send sequence incremented: 0 -> 1
[+] Record sealed successfully
```

```
=====
OPENING RECORD
-----
Received record length: 74 bytes
Expected receive sequence: 0
=====
[*] Parsed header
    Version: 1
    Direction: 1
    Sequence: 0
    Message type: 1
    Ciphertext length: 11 bytes
[+] Protocol version valid
[+] Record direction valid
[+] Sequence number valid
[+] IV valid
[+] HMAC verified successfully
[*] Receive sequence incremented: 0 -> 1
[+] Record opened successfully
```

Task 4 - Adversarial tests and security note

Automated Tests

The adversarial tests are implemented using pytest. The tests cover a variety of attacks, including MITM, a replay, and reflection attacks.

1. A valid handshake and bidirectional message
2. Modified ciphertext
3. Modified authenticated header
4. Replayed record
5. Record reflected into the opposite direction
6. Incorrect RSA public key

My implementation passes all six tests.

```
(.venv) jacob@hw1-agent:~/csce465-agentsec/hw2$ python -m pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/jacob/csce465-agentsec/hw2
collected 6 items

tests/test_handshake.py . [ 16%]
tests/test_secure_record.py ..... [100%]

===== 6 passed in 0.77s =====
```

Security Note

Attacker Model and Non-Goals

The primary goal for an attacker is to gain your trust or bypass authentication measures so they may read, alter, or send you communications. They can achieve this by posing as a trusted peer or tampering with the communication between endpoints. Attackers will attempt to insert themselves in between two endpoints, and observe or modify their communications, replay messages, or reflect messages back to the sender. After the session keys are derived, confidentiality of traffic is provided through encryption and the integrity of the data is maintained through authentication.

This implementation scopes itself at establishing a secure, authenticated channel between two parties. An endpoint may still be compromised, or a trusted connection with an attacker may be established, and thus the data sent over this protocol may contain malicious commands. This implementation does not attempt to solve these vulnerabilities.

Why CTR Requires Unique IVs

In AES-CTR, using the same key will generate the same keystream. This can lead to attackers learning the plaintext in some situations. We force a unique IV by deriving it from the session ID and sequence number. This means no two records will reuse the same key, preventing this vulnerability.

Why Verify-Before-Decrypt Matters

If an attacker modifies a record in any way, this record should be rejected. Thankfully, a modified record can be validated at the endpoint by verifying the HMAC tag. The ciphertext should be treated as untrusted, so this validation step should happen before decrypting the ciphertext and treating the potential attacker-controlled data as valid. Some elaborate attacks can leverage the ability to leak information by having an endpoint decrypt certain ciphertexts. Verifying that a record is authenticated before decrypting mitigates this risk.

Why Independent Encryption and MAC Keys are Used

This protocol uses independent encryption and MAC keys because they serve different purposes. AES-CTR is used for encryption and confidentiality, while HMAC provides integrity and authentication. These are separate purposes in the protocol, and they should use separate keys. The protocol goes one step further and provides separate keys for each direction of encryption and MAC as well. This gives each key a single purpose and reduces key reuse.

Comparison with AES-GCM

This assignment was an implementation of Encrypt-then-MAC, which required handling the key derivation and record layer sequencing manually. AES-GCM is an AEAD, or Authenticated Encryption with Associated Data, mode which provides both encryption and verification at the same time. While both ultimately achieve confidentiality and integrity, AEAD modes reduce the amount of implementation and are designed as single-pass algorithms, whereas encrypt-then-MAC requires two steps: encrypt, then HMAC. However, Encrypt-then-MAC maintains one advantage over AEAD modes due to the separated steps, an invalid record may be rejected before decrypting the ciphertext.

Why a Valid Protected Record May Still Contain a Dangerous Authorized Tool Call

Even after authenticating a record and validating the message has not been altered, a message's contents may still be malicious. Authentication is only one part of the puzzle. It can tell us a message is coming from where it says it is, a person is who they say they are, and that the data has remained intact; however, connections between bad actors may still be established over this protocol. This can be done by compromising a trusted peer, posing as an innocent party, or through more elaborate schemes. An attacker only has to find the weakest link, and that often isn't the encryption and authentication mechanism.